

SWIFT BENGALURU MEETUP



Build Your Own *Xcode Plugin.*

Make Xcode do the boring parts for you.

Kaveri Patil • iOS Developer

Swift Package Plugins • Hands-on • Real code

Quick show of hands.

This week, how many times did you...



`copy-paste a View + ViewModel + Tests skeleton for a new screen?`



`leave the same code-review comment you've left 40 times before?`



`run the same pre-commit script by hand because nobody set up the hook?`



`wish SwiftLint had that one rule for your team's style?`



THE ANSWER

Swift Package Plugins.

Tiny Swift programs that run inside your build workflow. Shipped as a Swift package. Versioned with your code. No external runtime. No extra install.

5.6+

Swift version where plugins landed (2022)

2

plugin types: build-tool & command

0

external tools or Ruby runtimes

Why not just use a script?

Because the automation lives outside the thing it automates.

Approach	Problem
Run Script build phase	Invisible in repo, breaks across machines
Makefile / shell	Teammates forget to run it
Fastlane / Ruby	Whole separate runtime to maintain
Pre-commit hook	Not installed automatically on clone
Swift Package Plugin	✓ Versioned with code. Runs automatically.

Two flavors.

The only question you'll ever need: when do I want this to run?



Build Tool Plugin

Runs every build. Automatic.

- ✓ Codegen (SwiftGen, Protobuf)
- ✓ Custom linters, Xcode warnings
- ✗ No source-tree writes
- ✗ No network



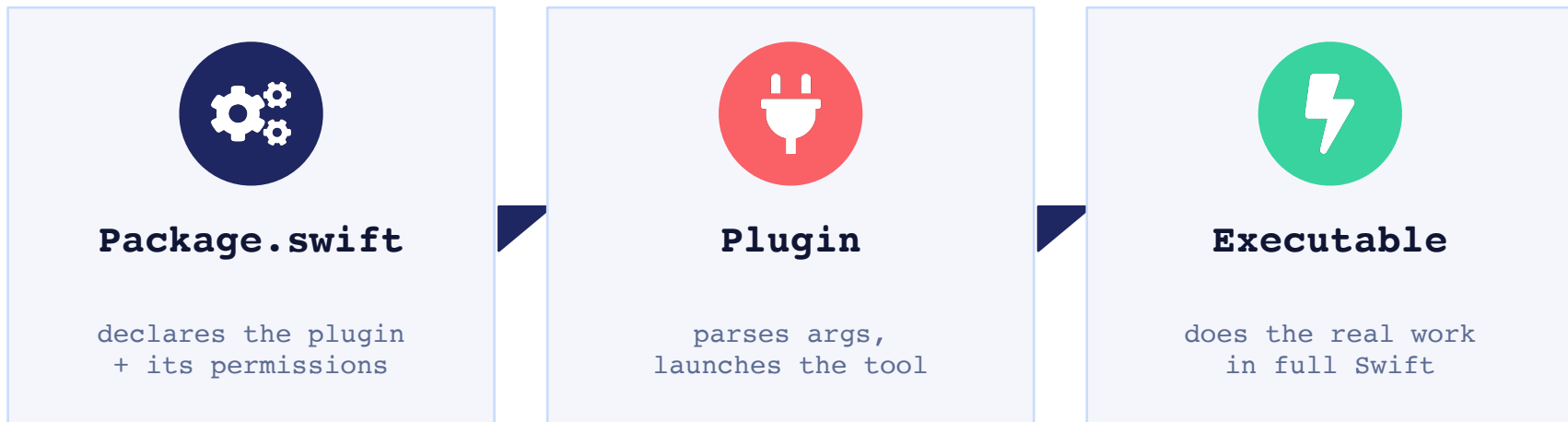
Command Plugin

Runs on demand. Terminal or Xcode.

- ✓ Scaffolders, formatters, generators
- ✓ Can request write access
- ✓ Can request network (SPM 5.9+)
- User-invoked, more powerful

Anatomy of a plugin.

Three pieces. Clean separation of concerns.



The plugin schedules. The executable does. The sandbox is what keeps the plugin honest.

How it all flows.

From command to generated files.



Total: ~80 lines of Swift for a working scaffolder.

Command plugin: the three knobs.

Verb, permissions, context. That's the whole API.



VERB

What users type

```
swift package  
generate-swift-model
```

kebab-case,
unique per package,
appears in Xcode menu.



PERMISSIONS

What it can do

```
.writeToPackage  
Directory  
.allowNetwork  
Connections
```

Declared in manifest.
User approves once.



CONTEXT

What it knows

```
context.package  
.directory  
context.tool  
(named: ...)
```

Passed to
performCommand().

1. Declare the plugin.

In Package.swift. This is metadata, not code that runs.

```
.plugin(  
  name: "GenerateSwiftModel",  
  capability: .command(  
    intent: .custom(  
      verb: "generate-swift-model",  
      description: "Generate Swift Codable structs from a JSON file"  
    ),  
    permissions: [  
      .writeToPackageDirectory(  
        reason: "Writes the generated .swift model file into your project Sources"  
      )  
    ]  
  ),  
  dependencies: ["JSONModelGeneratorCLI"]  
)
```

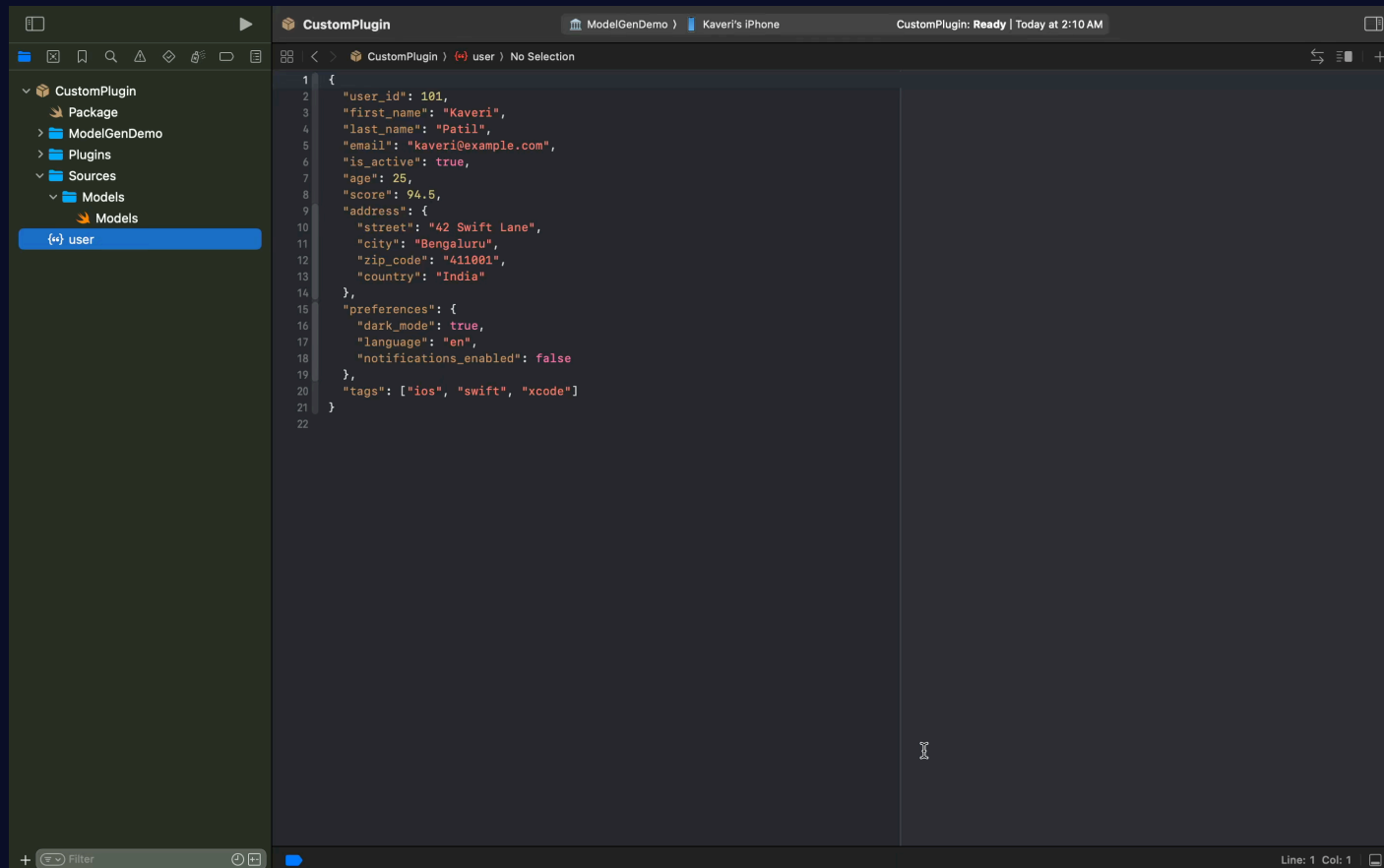
2. The plugin itself.

```
import PackagePlugin
import Foundation

@main
struct GenerateSwiftModel: CommandPlugin {
    func performCommand(context: PluginContext,
                        arguments: [String]) async throws {

        // 1. Parse the arguments the user typed
        // 2. Find the executable that does the real work
        // 3. Launch it, passing the args along
        // 4. Report any errors back to Xcode

    }
}
```



From empty folder to scaffolded module — in 60 seconds.

NOW — THE UNCOMFORTABLE QUESTION

"In 2026,
can't I just tell AI
to make me a feature module?"

Fair question. Let's reframe it.

They solve different problems.

And the best workflows use both.



Swift Package Plugin

Deterministic · Versioned · Offline

- Same input, same output, always
- Every clone of repo gets it free
- Runs in CI without AI or API keys
- Best for rules you can write down



Claude Code Plugin

Flexible · Judgment · Conversational

- Adapts to ambiguous requests
- Handles "also wire it to the tab bar"
- Reviews, explains, refactors with context
- Best for tasks needing interpretation

The real magic: AI decides *what* to build. Your plugin ensures *how* it's built.

Take home.

01 Plugins are just Swift code.

No Ruby. No extra runtime. Your team can read, review, and test them.

02 Build-tool vs Command: one question.

Does it need to run every build? Build-tool. Otherwise Command.

03 AI and plugins aren't rivals.

AI for judgment. Plugins for determinism. Compose them.

Thank you — questions?

Kaveri Patil



Thank you — Swift Bengaluru!